Preguntas de la ronda abierta — Entrega 2 (DSY1104)
0. Ficha rápida (repasar 10 minutos antes de entrar)
1. Diseño del proyecto con framework (IE2.1.1 / IE2.1.3)
2. Componentes React, props, estado y diseño responsivo (IE2.1.2 / IE2.1.4)
3. Pruebas unitarias con Jasmine y Karma (IE2.2.1 / IE2.2.2)
4. Proceso de testeo: entorno, mocks, análisis y cobertura (IE2.3.1 / IE2.3.2)
5. Integración con el backend y la base de datos
6. GitHub y proceso de desarrollo
7. Lo que quedó fuera y por qué
8. Preguntas incómodas: debilidades reales del código
9. Frases que conviene usar en la ronda
10. Checklist antes de presentar

Preguntas de la ronda abierta — Entrega 2 (DSY1104)

Guía de preparación para la ronda de preguntas del docente. Cada pregunta está ligada a un indicador de la rúbrica, con la respuesta que corresponde a lo que el proyecto **realmente** hace.

---

0. Ficha rápida (repasar 10 minutos antes de entrar)

Stack

Parte	Herramienta	Versión
Frontend	React + React Router	19.3 / 7.18
Bundler dev	Vite	8.3
Backend	Express	5.3
Base de datos	SQLite con `better-sqlite3`	13.0
Pruebas	Jasmine + Karma + Chrome Headless	7.0 / 6.4
Empaquetado de tests	Webpack + Babel	5.112 / 8.0
DOM en tests	Testing Library React	16.3
Lint	oxlint	1.88

Números que conviene saber de memoria

- **15 pruebas** unitarias en **6 archivos** de especificación.
- Cobertura: **61,03%** statements, **58,44%** branches, **47,12%** functions, **60,3%** lines.
- Base de datos: **3 tablas** (`productos`, `pedidos`, `pedido_detalle`), **20 productos**, **18 pedidos** de ejemplo, **532 unidades**.
- **11 endpoints** REST bajo `/api`.
- **4 vistas** del panel: Resumen, Productos, Costos, Pedidos.

Comandos

```
npm run dev        # vite + servidor de la API juntos
npm test           # corre las 15 pruebas y muestra la cobertura
npm run build      # build de producción
npm start          # sirve dist/ + API en el puerto 3000
```

Mapa de carpetas (preguntas tipo "muéstrame la estructura")

```
src/Components/
  Home/  Navbar/  ProductList/  DetalleProducto/  Cart/
CartContext/  Search/   ← tienda
  Productos/productos.json + consultaProductos.js
← datos
  Admin/  Admin.jsx, Dashboard, AdminProductos, AdminCostos,
AdminPedidos, api.js
server/
  index.js, db.js, datos.js,
rutas/{productos,costos,pedidos,resumen}.js
tests/  6 archivos .spec.jsx
```

---

1. Diseño del proyecto con framework (IE2.1.1 / IE2.1.3)

P1. ¿Por qué React y no seguir con el HTML/CSS/JS de la plantilla? La plantilla traía el carrito hecho en JavaScript plano, con el estado repartido en variables del DOM. Al migrar a React el carrito pasó a ser estado de un componente (`CartContext`), el DOM se genera desde el estado y las vistas se pueden componer. Con JS plano cada cambio de vista era reescritura manual del HTML; con React es un cambio de estado.

P2. ¿Qué es un componente en tu proyecto y cómo decides dónde cortar? Un componente es una parte de la interfaz con una sola responsabilidad. En este proyecto hay componentes de la tienda (`ProductList`, `DetalleProducto`, `Cart`, `Search`), el contexto del carrito y los del panel (`Dashboard`, `AdminProductos`, `AdminCostos`, `AdminPedidos`). El criterio de corte que usé es el que pide el anexo: un componente con **una** responsabilidad, que recibe datos por **props** y que maneja estado propio solo si lo necesita.

P3. ¿Por qué separaste `Admin` del resto de la tienda? El anexo pide "vistas adicionales" como panel de administración. Es una sección con navegación y flujo propios, así que quedó en `/admin` con rutas anidadas de React Router (`Admin` es el layout y usa `<Outlet />` para las sub-vistas). No tocó el código de la tienda: el enlace al panel se agregó al Navbar y las rutas en `App.jsx`.

P4. ¿Qué ventaja te dio usar React Router? Permite rutas declarativas y anidadas. El panel son cuatro rutas dentro de un layout común, y el carrito, el detalle de producto y la búsqueda son rutas distintas con su propio `Link`. También permite abrir directo `/admin/productos` y que funcione al recargar.

P5. ¿Cómo se comunica la tienda con el panel? No se comunican entre sí: ambos hablando con la misma API. La tienda lee `/api/productos` y el panel escribe en la misma tabla. Por eso, si en el panel dejo un producto sin stock, la tienda deja de mostrarlo sin borrar el producto de la base.

P6. ¿Qué pasa si el servidor de la API está apagado y entro a la tienda? `consultaProductos()` hace `fetch` a `/api/productos` y, si la respuesta falla, cae al `productos.json` local como respaldo. La tienda nunca queda en blanco. El panel, en cambio, sí necesita el servidor: es donde se administer la data.

---

2. Componentes React, props, estado y diseño responsivo (IE2.1.2 / IE2.1.4)

P7. ¿Qué diferencia hay entre props y estado en tu proyecto? Props son datos que el padre pasa al hijo y el hijo no puede modificar. Estado es dato que el componente maneja por dentro. Ejemplo: `ProductList` recibe `buscarTermino` por prop yInternamente maneja `orden` y `filtros` con `useState`. `Search` recibe `onSearch` por prop y el término que escribe en `useState`.

P8. ¿Por qué el carrito está en un Context y no como prop? Porque lo usan `Navbar` (el contador), `DetalleProducto` (agregar) y `Cart` (listar, cambiar cantidad, eliminar), y esos componentes están en ramas distintas del árbol. Pasarlo por props por todo el árbol sería acoplarlos. `CartContext` con `useCart()` expone el estado y las acciones.

P9. Muéstrame cómo manejas el estado en un formulario. En el formulario del panel: un solo `useState` con un objeto `formulario` y un `handleChange` genérico que hace `{...formulario, [name]: value}`. Cada campo tiene `name` y el `onChange` lee `e.target.name`. Es más simple que un `useState` por campo.

P10. ¿Cómo implementaste el diseño responsivo con Bootstrap? Bootstrap entra por CDN y aporta el grid (`container`, `row`, `col-12`, `col-md-3`, `col-lg-2`), las tablas y los formularios del panel. El CSS se carga **dinámicamente** al entrar a `/admin`: en el `useEffect` de `Admin.jsx` se agrega el `<link>` al

`document.head` y al salir se quita. Lo hice así a propósito: el *reboot* de Bootstrap pisa los estilos base de la tienda (la tipografía del `body`, los márgenes de los `p`), y así la tienda quedó idéntica a como estaba. Además el `body` recibe la clase `modo-admin` para que el panel use el ancho completo.

P11. ¿Y la tienda entonces no usa Bootstrap? La tienda conserva su CSS propio, que ya tenía sus *media queries* para móvil, tablet y escritorio (`.product-list` cambia de columnas en 1024px, 768px y 480px). El uso de Bootstrap como framework de CSS del proyecto está en la sección de administración, que es la parte nueva que construí. En el anexo se pedía Bootstrap para asegurar responsividad, y la parte nueva lo cumple.

P12. ¿Cómo verificaste que se ve bien en distintos tamaños? Revisé el panel en 1280px, en 1024px y en móvil de 390px. En móvil el menú lateral pasa a ser horizontal y el `body` vuelve a su ancho normal. Comprobé también que no apareciera scroll horizontal en ninguno de los tamaños.

---

3. Pruebas unitarias con Jasmine y Karma (IE2.2.1 / IE2.2.2)

P13. ¿Por qué Jasmine y Karma, y no Jest? Es lo que pide la rúbrica de forma explícita. Jasmine es el framework de aserciones (`describe` / `it` / `expect`) y Karma es el runner que ejecuta las pruebas en un navegador real, levantando un servidor, cargando el bundle y conectando el navegador por socket. La diferencia con Jest es que Karma no trae un DOM: hay que cargar el framework, el adapter y el bundle a mano.

P14. ¿Cómo configuraste el entorno de pruebas? `karma.conf.cjs` define el framework (`jasmine` + `webpack`), los archivos (`tests/**/*.spec.jsx`), el preprocesador de Webpack y el launcher `ChromeHeadless`. `babel.config.cjs` transpila JSX y los `import` para el navegador, y `karma.instrumentador.cjs` instrumenta los archivos de `src/` para medir la cobertura con `istanbul-lib-instrument`.

P15. ¿Por qué Webpack y Babel si el proyecto usa Vite? Porque Karma no entiende Vite: el plugin de Vite está pensado para el dev server y para el build, no para servir un bundle a un runner externo. Karma necesita su propio pipeline de empaquetado, así que ahí uso Webpack, que es lo estándar cuando se usa Karma. Son dos pipelines distintos y para propósito distintos.

P16. ¿Cómo ejecutas las pruebas y qué muestran? `npm test` levanta Karma, que empaqueta las pruebas con Webpack, abre Chrome Headless, ejecuta los 15 casos y muestra el resultado. Con el flag de cobertura genera además el resumen por métrica y el reporte HTML en `coverage/index.html`.

P17. ¿Qué verifican tus 15 pruebas?

Archivo	Casos	Qué verifica
`ProductList.spec.jsx`	5	lista los 20 productos, filtra por categoría, ordena por precio, busca por nombre y **oculta los que quedaron sin stock**
`CartContext.spec.jsx`	3	agregar, sumar cantidad si ya existe, eliminar
`Cart.spec.jsx`	2	carrito vacío y cálculo de subtotal, envío y total
`DetalleProducto.spec.jsx`	2	muestra los datos según el id de la URL y avisa cuando no existe
`Search.spec.jsx`	1	avisa el término que se escribe
`AdminProductos.spec.jsx`	2	carga los productos desde la API y muestra el total del listado

P18. ¿Manejas el DOM en las pruebas? Sí. Con Testing Library consulto el DOM por lo que ve el usuario: busco el texto del producto, los `input` por su `name` y los botones por su etiqueta. En `ProductList` cuento las tarjetas con `document.querySelectorAll(".product-card")` y en `Cart` verifico el valor del input de cantidad después de hacer clic en “+”.

P19. ¿Qué tipos de pruebas hiciste según la propuesta del anexo? Las cuatro que propone el anexo: - **Renderizado correcto**: la lista renderiza los 20 elementos del conjunto de datos. - **Renderizado condicional**: el mensaje de error solo aparece cuando hay error (en `AdminProductos` mockeo un `fetch` que falla y compruebo que aparece la caja de error), y el producto sin stock no se renderiza. - **Props**: `Search` recibe `onSearch` y el botón de “Añadir al carrito” dispara la función que le pasa el padre. - **Estado**: el estado del formulario cambia cuando el usuario escribe, y el clic en el botón “agregar al carrito” cambia el estado del carrito. - **Eventos**: se simulan clics con `fireEvent.click` y se comprueba que el estado cambie.

P20. ¿Por qué en `DetalleProducto` no mockeaste `useParams`? Porque con `MemoryRouter` puedo renderizar la ruta real (`/producto/:id`) con la URL que quiero probar. Es una prueba más realista que mockear el hook: se verifica la integración con React Router de verdad.

---

4. Proceso de testeo: entorno, mocks, análisis y cobertura (IE2.3.1 / IE2.3.2)

P21. ¿Qué herramientas usaste y para qué? Jasmine para escribir y ejecutar las aserciones, Karma para correrlas en el navegador, Webpack + Babel para armar el bundle, Chrome Headless como navegador y Testing Library para consultar el DOM. Para la cobertura, `istanbul-lib-instrument` instrumenta los archivos de `src/` y `karma-coverage` arma el reporte.

P22. ¿Por qué mockear el `fetch` y no levantar la base de datos en las pruebas? Para que las pruebas sean unitarias: el componente se prueba aislado del servidor. En `AdminProductos.spec.jsx` hago `spyOn(window, "fetch").and.returnValue(...)` y devuelvo una respuesta falsa. La prueba no depende de que exista la API ni de los datos reales, y puedo probar casos como "la API falla" que en la vida real son difíciles de provocar.

P23. ¿Qué otros mocks usaste? En `Search.spec.jsx` uso `jasmine.createSpy("onSearch")` para comprobar que se avisa el término. Y en `ProductList.spec.jsx` mockeo `fetch` para el caso de stock cero. Los spies de Jasmine se restauran solos después de cada caso, así que no se contaminan entre pruebas.

P24. ¿Qué es una prueba de integración y tú haces de ese tipo? Una prueba de integración verifica que dos piezas funcionan juntas. Mis 15 pruebas son unitarias: se prueba un componente o una función aislada. La integración entre el frontend y la API la verifiqué de otra forma: levantando el proyecto completo y probando el flujo real en el navegador (crear, editar y eliminar un producto contra SQLite, y comprobar que al dejar un producto sin stock desaparece de la tienda). Eso no es una prueba automatizada, es verificación manual, y es una limitación que tengo.

P25. ¿Cómo mides la cobertura y qué significa? La cobertura se mide instrumentando el código: en `karma.instrumentador.cjs` se usa `istanbul-lib-instrument` para contar qué líneas, ramas y funciones se ejecutan durante las pruebas. El resultado va a `window.__coverage__`, Karma lo recoge y `karma-coverage` genera el reporte HTML. El número no dice que el código esté bien, solo qué partes se ejercitan: una función con 0% puede estar bien o puede estar rota.

P26. ¿Qué muestra el resultado actual y cómo lo interpretas? 61,03% de statements, 58,44% de branches, 47,12% de functions y 60,3% de lines. Las partes más cubiertas son el contexto del carrito, `ProductList` y `DetalleProducto`. Las menos cubiertos son los formularios del panel (`AdminProductos`, `AdminCostos`) y `AdminPedidos`, porque ahí falta probar el alta, la edición y el borrado. Es el siguiente paso que tomaría.

P27. ¿Qué harías para subir la cobertura? Probar los formularios del panel: enviar el alta de un producto, verificar que llama a la API con el cuerpo correcto, probar la edición y la eliminación. Y probar `AdminCostos` y `AdminPedidos`, que hoy solo se renderizan. No todos los puntos de cobertura valen lo mismo: prefiero un 60% con los flujos críticos probados que un 90% con getters.

---

5. Integración con el backend y la base de datos

P28. ¿Por qué SQLite y no otra base? Porque el enunciado pide una base de datos y SQLite no necesita instalar ni configurar un servidor: es un archivo. Con `better-sqlite3` la API es síncrona, que en Express queda mucho más simple que las promesas. El archivo queda en `database/tienditagaming.db` y está en `.gitignore`.

P29. ¿Cómo se crea la base? En `server/db.js` se abre el archivo y se ejecutan los `CREATE TABLE IF NOT EXISTS`. En `server/datos.js` hay funciones `sembrarProductos()` y `sembrarPedidos()` que insertan los datos de ejemplo **solo si la tabla está vacía**, así que en cada arranque no se duplica nada.

P30. ¿Qué tablas tienes y cómo se relacionan? Tres: `productos`, `pedidos` y `pedido_detalle`. `pedido_detalle` tiene `pedido_id` y `producto_id`, así que un pedido puede tener varias líneas y un producto estar en varios pedidos. La línea guarda el precio y el costo unitarios al momento de la venta, para que un cambio de precio después no altere la historia.

P31. ¿Cómo se actualiza la interfaz cuando cambia algo en la base? El panel pide los datos a la API después de cada operación (crear, editar, borrar, cambiar estado) y los setea en el estado del componente, así que React re-renderiza la tabla. Del lado de la tienda, la lista se carga al montar el componente; por eso al navegar de vuelta a la tienda ya aparece el cambio.

P32. ¿Por qué el costo no está en el JSON de productos? El JSON es el catálogo inicial de la tienda. El costo es un dato interno de administración: no tiene sentido mostrarlo al cliente. En la tabla `productos` hay una columna `costo` que se calcula al sembrar, con un factor por categoría (72% en consolas, 58% en periféricos, 78% en videojuegos).

---

6. GitHub y proceso de desarrollo

P33. ¿Cómo se organiza el trabajo en GitHub? Cada parte de la entrega tiene su rama y su commit con un mensaje claro: la migración a React, la incorporación de las pruebas, el panel de administración y la integración con SQLite. El `.gitignore` excluye `node_modules`, `dist`, la carpeta `database/` y `coverage/`, que son artefactos locales.

P34. ¿Qué se sube y qué no? Se sube el código fuente, los tests, los archivos de configuración y el README. No se suben `node_modules` (se reinstala con `npm install`), ni `dist` (se genera con `npm run build`), ni la base de datos (se regenera sola al levantar el servidor con los datos de ejemplo), ni el reporte de cobertura.

P35. ¿Qué cambiarías de tu propio flujo de trabajo? Añadir un commit por cambio pequeño en vez de agrupar por entrega, y escribir la prueba junto al componente, no al final. Eso detectaría antes los errores de integración.

> Nota: **el repositorio todavía no está inicializado.** Es el entregable "Enlace GitHub público del proyecto frontend" y hay que crearlo antes de la entrega.

---

7. Lo que quedó fuera y por qué

P36. ¿Qué vistas del diagrama de flujo no implementaste? Del diagrama de la tienda (Figura 3 del anexo) quedaron fuera: registro/inicio de sesión, "Mis cuentas", "Nosotros", "Contacto" y el historial de compras del cliente. Implementé las que el anexo nombra como vistas nuevas: **Categorías**, **Ofertas** y el flujo de **Compra** con checkout, pago correcto y pago con error. Del panel administrativo (Figura 10) quedaron fuera Reportes, gestión de usuarios y Perfil; implementé Resumen, Productos, Costos y Pedidos con su detalle.

P37. ¿Por qué no las implementaste todas? Por alcance y por tiempo. El anexo dice que los contenidos son sugerencias y que se priorice un sistema funcional. Prefiero entregar pocos flujos completos y probados que muchos a medias. Si tuviera que continuar, el siguiente sería Reportes en el panel, porque con los datos de `pedidos` y `pedido_detalle` se pueden sacar ventas por mes, productos más vendidos y margen por categoría.

P38. ¿Qué harías distinto si tuvieras más tiempo? Agregar autenticación (hoy el panel es accesible sin protección), persistir el carrito en `localStorage` para que no se pierda al recargar, y subir la cobertura de los formularios del panel. También automatizar las pruebas de integración contra la API real con `supertest`.

P39. ¿Qué error cometiste durante el desarrollo que te costó tiempo? El más tiempo me costó el tema del `act` de React 19 en las pruebas: al principio las pruebas de `ProductList` fallaban porque la actualización de estado quedaba encolada en la cola de `act` y el DOM nunca cambiaba, aunque los datos llegaran bien. Lo resolví envolviendo el render en `await act(async () => {...})` y mockeando el `fetch`. La lección: en React 19 hay que ser explícito con el alcance de `act` cuando el componente carga datos de forma asíncrona.

---

8. Preguntas incómodas: debilidades reales del código

Estas son cosas que un profe revisando el código **sí** va a encontrar. Conviene tener la respuesta clara: reconocer el problema y explicar el arreglo.

P40. En `CartContext` haces `carritoActualizado[exiteProducto].cantidad += 1`. Eso muta el estado, ¿no? Correcto, es una mutación directa del objeto que está en el estado.Funciona porque el array es nuevo, pero el objeto interno es el mismo. La forma correcta es crear el objeto nuevo: `{...producto, cantidad: producto.cantidad + 1}` dentro de un `map`. Lo dejé así porque venía del código base, pero es un pendiente que anoté para corregir.

P41. En `ProductList` declaras `const [error] = useState(null)` y nunca cambias el error. Efectivamente, ese estado quedó de la migración y no se usa: el componente siempre entra por la rama de los productos. Debería settar el error cuando la consulta a la API falle, para poder mostrar el mensaje en vez de una lista vacía. Es código muerto que hay que limpiar.

P42. En `AdminPedidos` hay un `map` anidado y React complains de una `key` faltante. Sí, es un aviso de React en la consola. Es la tabla de detalle del pedido, que se arma con dos `map` anidados y se me olvidó la `key` en el interno. La solución es usar el índice o el id de la línea como `key`.

P43. Usaste `var` en algunos lugares del panel en vez de `let` / `const`. Sí, se me escapó en `AdminProductos` y `AdminCostos`. `var` tiene hoisting y scope de función, `const` / `let` tienen scope de bloque. No rompe nada ahí, pero es mejor usar `const` / `let` siempre.

P44. Dejaste un `console.log` en `AdminProductos`. Sí, se me quedó uno después de probar el alta de productos. Sirve de evidencia de que la acción se ejecutó, pero en un proyecto lo correcto es sacarlo o usar un logger con niveles.

P45. En el fixture de `AdminProductos.spec.jsx` escribiste `preco` en vez de `precio`. Es un typo en un dato de prueba, la propiedad sobra y no afecta nada porque el mock se usa tal cual. Está ahí porque el archivo lo escribí rápido.

P46. En `Cart.css` tienes `grid-template-columns: 2f 1fr 1fr 1fr 1fr`. Es un typo, debería ser `2fr`. En la vista de escritorio no se nota porque el grid ignora la unidad inválida y reparte las columnas, pero en móvil la tabla se distribuye distinto. Es un error que viene del código base.

P47. En `ProductList` el `alt` de la imagen es `alt={producto.image}`. Sí, ahí debería ser `alt={producto.nombre}`. `producto.image` no existe en los datos, así que el atributo queda con el valor `undefined` y el lector de pantalla no anuncia nada. Es un error de accesibilidad que conviene corregir.

P48. El costo de envío está fijo en 10.000 en `Cart.jsx`. Sí, es una constante dentro del componente. Lo correcto sería moverla a una configuración o a la API, porque hoy no se puede cambiar sin tocar el código. En la versión de checkout del anexo sí hay más opciones de envío.

P49. La base de datos se regenera con datos de ejemplo cada vez. No: solo se siembra si la tabla está vacía. La condición `if (cantidad > 0) return` lo evita. El archivo está en `.gitignore`, así que en el repositorio no viaja la base, solo el código que la crea.

P50. ¿Y si dos personas trabajan en el panel a la vez? Hoy no hay control de concurrencia: si dos editan el mismo producto al mismo tiempo, gana la última escritura. Como no hay autenticación, tampoco hay auditoría de quién cambió qué. Con un campo `actualizado_por` y una revisión de la versión se resolvería.

---

9. Frases que conviene usar en la ronda

- "Partí de la plantilla y la fui migrando por partes, no la reescribí de una."
- "Cada componente tiene una sola responsabilidad; cuando un componente empezaba a hacer dos cosas, lo partía en otro."
- "Las pruebas son unitarias: aíslo el componente y mockeo lo que depende de fuera, como el `fetch`."
- "La cobertura me sirve para ver qué partes no están tocadas por las pruebas, no como métrica de calidad."
- "Cuando el panel cambia el stock, la tienda lo refleja porque ambos leen de la misma API."
- "Para no romper el diseño de la tienda, cargué Bootstrap dinámicamente solo en la sección de administración."

---

10. Checklist antes de presentar

- [ ] `npm install` limpio y `npm test` en verde (15/15).
- [ ] `npm run build` sin errores.
- [ ] `npm run dev` levanta la tienda y el panel, y la base se crea sola.
- [ ] El repositorio de GitHub está inicializado, con commits y publicado.
- [ ] El ZIP del proyecto está listo (sin `node_modules`, `dist`, `database/`, `coverage/`).
- [ ] Documento ERS V2 y documento de cobertura de testing listos.
- [ ] Repasar la sección 8: son las preguntas que más pesan.